

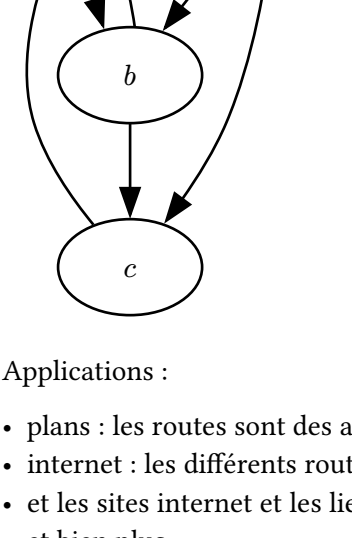
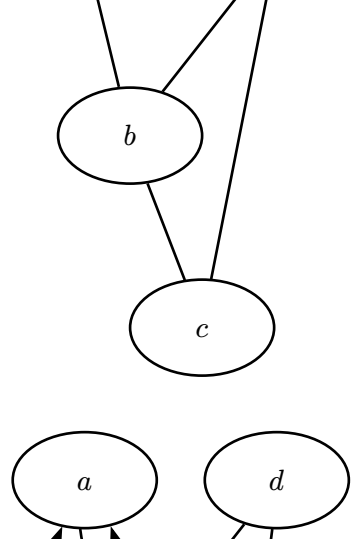
I) Pourquoi, comment ?

Définition 1 [Graphe]

Un graphe simple est une paire (S, E) , avec S l'ensemble des sommets, et E l'ensemble des arrêtes.

Un graphe est dit *orienté* si les arrêtes ont une direction, et E est alors un sous-ensemble des paires d'éléments de S .

Sinon, le graphe est *non-orienté*, et $E \subset P_2(S)$ les parties à deux éléments de S .



Applications :

- plans : les routes sont des arrêtes, les intersection sont des sommets
 - internet : les différents routeurs, serveurs, connexions forment un (très) grand graphe
 - et les sites internet et les liens qui les relient aussi !
 - et bien plus.
- Dans tous ces cas, on aimerait pouvoir
- avoir accès aux composantes connexes du graphe (les parties reliées entre elles)
 - avoir accès au plus cours chemin entre deux points
 - explorer une partie du graphe (par exemple, un crawler web)

II) Les parcours

II.1) Anatomie d'un parcours

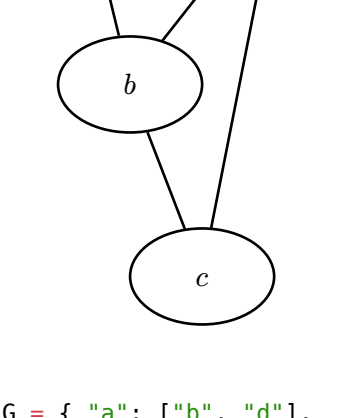
On a besoin :

- d'une structure de donnée pour stocker des noeuds

PARCOURS D'UN GRAPHE	
Entrée : G un graphe, d un élément de départ	
1	Ajouter d dans la structure
2	Tant que la structure n'est pas vide
3	Prendre x un élément de la structure
4	(...)
5	Ajouter ses voisins non visités à la structure.

II.2) Parcours en profondeur : la pile

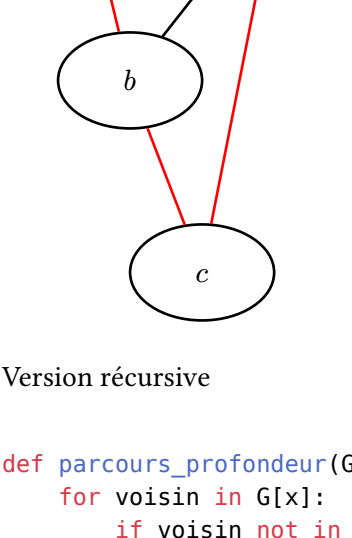
Voici une implem en python, qui imprime simplement les sommets dans l'ordre. On suppose que le graphe est implémenté par un dictionnaire de listes d'adjacence :



```
G = {
    "a": ["b", "d"],
    "b": ["d", "a", "c"],
    "d": ["b"],
    "c": ["b", "a"]}

def parcours_profondeur(G, d):
    pile = [d]
    while len(pile) > 0:
        ...
        x = pile.pop()
        print(x)
        for voisin in G[x]:
            pile.append(voisin)
```

```
def parcours_profondeur(G, d):
    pile = [d]
    deja_vu = {d: True}
    while len(pile) > 0:
        x = pile.pop()
        print(x)
        # On ajoute les voisins à la pile
        for voisin in G[x]:
            if voisin not in deja_vu:
                pile.append(voisin)
                #Evite d'ajouter les mêmes sommets à la pile plusieurs fois
                deja_vu[voisin] = True
```



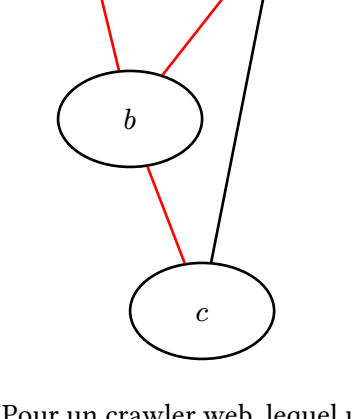
Version récursive

```
def parcours_profondeur(G, d, deja_vu):
    for voisin in G[d]:
        if voisin not in deja_vu:
            deja_vu[voisin] = True
            parcours_profondeur(G, voisin, deja_vu)
```

II.3) Parcours en largeur

```
from collections import deque

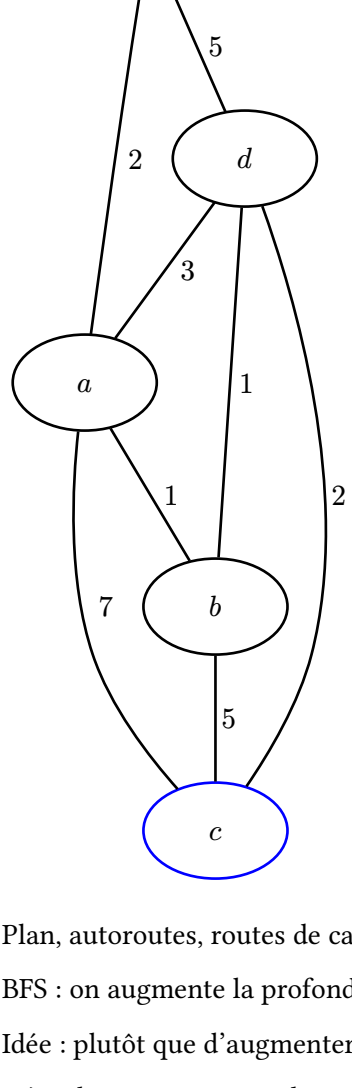
def parcours_profondeur(G, d):
    file = deque()
    file.appendleft(d)
    deja_vu = {d: True}
    while len(pile) > 0:
        x = pile.pop()
        print(x)
        # On ajoute les voisins à la pile
        for voisin in G[x]:
            if voisin not in deja_vu:
                pile.appendleft(voisin)
                #Evite d'ajouter les mêmes sommets à la pile plusieurs fois
                deja_vu[voisin] = True
```



Pour un crawler web, lequel utiliseriez-vous ? Et pour trouver la sortie d'un labyrinthe ?

III) Plus court chemin

Sur un graphe non pondéré, pouvez-vous faire un parcours qui permet ça ?

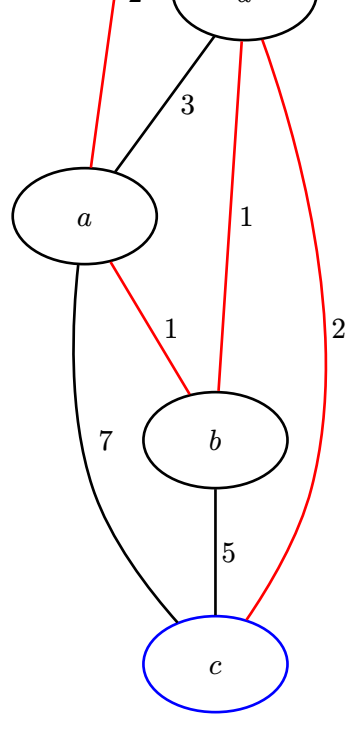


Plan, autoroutes, routes de campagnes, etc

BFS : on augmente la profondeur graduellement : d'abord 1 de distance, puis 2, etc.

Idee : plutôt que d'augmenter la profondeur, on augmente la pondération.

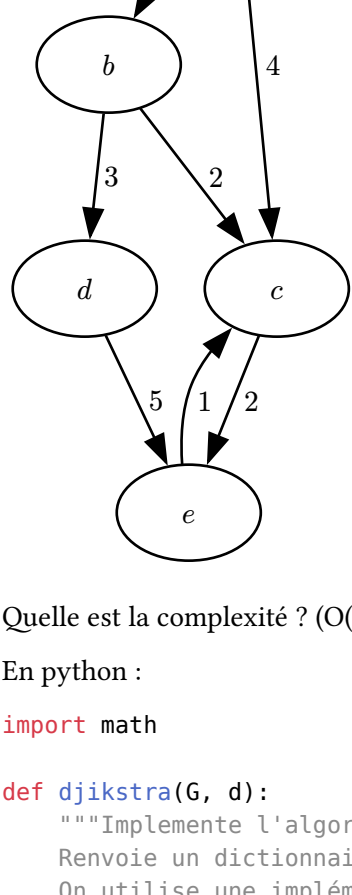
Déroulement sur exemple



III.1) Algorithme de Dijkstra

ALGORITHME DE DIJKSTRA	
Entrée : G un graphe connexe, d un sommet de départ	
Sortie : La distance de d à chaque sommet du graphe.	
1	Soit F une file de priorité, avec tous les noeuds associé à un poids +∞
2	Soit V un dictionnaire
3	Ajouter (d, 0) à F
4	Tant que F n'est pas vide
5	Prendre (x, l) le couple minimal de F
6	Ajouter (x, l) à V
7	Pour chaque arrête (x, y, ρ)
8	Si y n'est pas dans V
9	Si y n'est pas dans F
10	ajouter (y, l + ρ) F.
11	ajouter (y, l + ρ) F.
12	mettre à jour le poids de y dans F, au minimum entre le poids actuel et l + ρ
13	Renvoyer V

Dérouler sur l'exemple.



Quelle est la complexité ? (O(|E| + |S|) log |S|))

En python :

```
import math

def dijkstra(G, d):
    """Implemente l'algorithme de Dijkstra sur un graphe par liste d'adjacence.
    Renvoie un dictionnaire qui à chaque sommet associe sa distance à d.
    On utilise une implémentation naive de file de priorité.
    """
    noeuds = {s: math.inf for s in G.keys()}
    noeuds[d] = 0
    V = {}
    while len(noeuds) > 0:
        # On trouve le plus petit élément de noeuds
        sommet_min = None
        distance_min = math.inf
        for (sommet, distance) in noeuds.items():
            if distance < distance_min:
                distance_min = distance
                sommet_min = sommet

        # Si le sommet minimum est None, c'est qu'on a parcouru toute la composante
        # connexe
        # On peut donc renvoyer le dictionnaire;
        if sommet_min is None:
            return V

        # On le supprime de la liste des noeuds et on l'ajoute au dictionnaire
        # résultat.
        del noeuds[sommet_min]
        V[sommet_min] = distance_min

        # On met à jour la distance des voisins
        for (voisin, poids) in G[sommet_min]:
            if voisin not in V:
                noeuds[voisin] = min(noeuds[voisin], distance_min + poids)

    return V
```

Quelle est la complexité de cette implémentation ? -> O(|V|²)

<https://visualgo.net/>